INCREMENTAL CODE AUDIT

AsymptoticAnalysis

Target-dependent source shifts can invalidate exponential-core remainder bounds

Wolfram 15 and Mathics3: source analysis, native reproduction, a minimal repair, and independently checked mathematics

Principal result. A public call for the inverse of $e^x + 1$ returns the correct finite expression $\log y - y^{-1}$ but the incorrect remainder scale $e^{-2y}y^{-2}$. The true error is asymptotic to $\frac{1}{2}y^{-2}$. A one-line scope guard rejects the offending target-dependent chart without forbidding a legitimate target-dependent `CoreInverse`.

Repository	`VladimirReshetnikov/Asymptotic`
Source revision	`8cee870994f506b501bae3ea6bd4a3a7edb895c1`
Revision time	10 September 2026, 17:10:56 UTC
Native execution	Wolfram 15.0.0, Linux x86 (64-bit)
Mathics execution	Not performed; portable acceptance probe supplied
Date	10 September 2026

This report retains one newly reproduced correctness finding after comparison with the repository's recorded review mechanisms. It is not a restatement of the existing backlog, a full-suite acceptance record, or an assertion that every repository line has been reviewed.

Abstract

This audit investigates selected analytic engines, compatibility adapters, result operations, special-function formulas, and validation tooling at a pinned revision of AsymptoticAnalysis. The existing review register, wave-3 and wave-4 intakes, and wave-5 index are used to exclude previously recorded mechanisms. The retained new finding concerns the scope of the `SourceShift` option in `AsymptoticExponentialCoreInverse`. The constructor excludes the source variable from the shift but does not exclude the requested target variable. Consequently, an admitted change of source coordinates can turn a fixed core amplitude into a target-dependent amplitude. The coefficient recurrence still produces the right displayed approximation, while the fixed-data remainder argument silently treats an unbounded factor as a constant.

The failure is reproduced through the public API in Wolfram 15.0.0. An elementary exact inverse proves that the returned remainder is false, rather than merely conservative or insufficiently documented. An all-orders calculation identifies the missing amplitude factor and supplies explicit shift-independent bounds for a narrow exponential family. A minimal guard replacement is exercised on a downloaded temporary copy of the pinned standalone package; seven focused outcome classifications preserve the intended distinction between source shifts and supplied inverse expressions. Twenty independent Python test methods pass. The accompanying twelve-case MUnit file and portable smoke file are supplied for further acceptance, not represented as executed Mathics or full-suite evidence.

Contents

1 Scope, baseline, and novelty discipline

1.1 What was reviewed

The reviewed repository is `VladimirReshetnikov/Asymptotic`. The pinned source revision is `8cee870994f506b501bae3ea6bd4a3a7edb895c1`.
Its recorded commit time is 17:10:56 UTC on 10 September 2026. The native reproduction downloaded the standalone `AsymptoticAnalysis.wl` at that exact revision, rather than following a moving `main` URL. The downloaded entry contained 692,381 bytes and returned `Null` from `Get`; the expected public constructor was present. The repository identity and maintained entry points come from the pinned repository and its source map [1, 2]. The byte count and loading result are observations of this audit, transcribed in the accompanying evidence file.

The source pass included the exponential-core and Lambert parsers; Gamma, Barnes, Dirichlet, and parameterized special-function paths; source reconstruction; selected series arithmetic and refinement paths; the exact-interval certificate engine; and several Mathics-specific proof, algebra, calculus, simplification, and formatting adapters. A maintained Markdown-link validator was also inspected. Some files were read completely, while several larger files were inspected in selected ranges. `evidence/source_inventory.csv` records that distinction file by file. Loading a complete standalone package is not the same as auditing every source line.

The review question was not merely whether an expression evaluates. For each candidate, the important distinctions were whether a formula is exact, whether its coefficients are correct, whether its remainder theorem applies on the stored approach, and whether its proof assumptions survive coordinate changes. A successful symbolic calculation can establish the first two without establishing the latter two. The retained finding is a particularly small example of that separation.

1.2 How prior findings were excluded

The maintained status register organizes the earlier attributed findings and proposals. The wave-3 and wave-4 intake documents consolidate their mechanisms, and the wave-5 index maps the newest reports and retirement tombstones [3, 4, 5, 6]. These records were used as the exclusion set. The report does not reissue the already recorded option-routing, generic resource-budget, native-export, derivative-contract, numerical-precision, or validation-runner recommendations.

For the new finding, the closest recorded neighbors are C15, X10, report 43 F03, and W3-02. Their identifiers are retained in the novelty ledger so that the maintainer can check the distinction. The present witness does not compose two stored germs, does not inject an eliminated source variable through a supplied inverse, and does not depend on an option-name parser choosing the wrong key. It passes a recognized option directly to one specialized constructor, and that constructor installs the target inside the normalized core amplitude. The repair belongs at that particular scope boundary.

This is a mechanism-level comparison against the maintained register and intakes, not a claim to have reread every line of all archived articles. No matching recorded mechanism was found in those reviewed records. The novelty claim should be understood at that precise scope; the archive's unindexed prose could warrant an additional attribution check before publication upstream.

1.3 Evidence levels

This report keeps four kinds of evidence separate. Source inspection establishes the missing guard and the formulas that consume it. A fresh native execution establishes public reachability and the actual returned expression and error scale. An exact mathematical calculation establishes that the returned error scale is wrong. A patched native execution establishes only the stated focused outcomes, not global compatibility.

The Wolfram connector was initially unavailable and later returned successful results. Two subsequent larger attempts also returned HTTP 502. Those failed calls supply no observations. The successful calls identify the kernel as `15.0.0 for Linux x86 (64-bit) (May 6, 2026)`. There was no successful Mathics execution in this audit, and no claim is made for Wolfram 15.0.1 on Windows or for the full repository test suite.

2 N01: a public call returns an invalid remainder

2.1 The request

After loading the pinned package, the following request succeeds in the unmodified Wolfram execution:

```
s = AsymptoticAnalysis`AsymptoticExponentialCoreInverse[
  Exp[x], 1, {x, Infinity}, {y, 1},
  "SourceShift" -> -Abs[y]
];
```

The forward equation is simply

$$e^x + 1 = y, \qquad x \longrightarrow +\infty, \tag{1}$$

whose selected real inverse is

$$x_*(y) = \log(y - 1), \qquad y > 1. \tag{2}$$

No branch selection ambiguity, opaque special function, implicit numerical solver, or user-supplied input remainder is needed.

Under $y > 2$, the actual returned projections are

$$\mathrm{Normal}(s) = \log y - \frac{1}{y}, \tag{3}$$

$$s[\texttt{"RemainderScaleExpression"}] = \frac{e^{-2y}}{y^2}. \tag{4}$$

The public object is a `GeneralizedSeries`, not a failure or a native/formal object with an explicitly missing analytic remainder. Its normalized core parameters include

```
<|"a" -> Exp[-Abs[y]], "b" -> 0, "c" -> 1,
  "p" -> 1, "Offset" -> 0, "Exact" -> True|>
```

These outputs are preserved in `evidence/native_observations.json`. The source path responsible is `src/Kernel/ExponentialCorePerturbation.wl`, particularly the admission guard and the sector construction [7].

2.2 The unshifted control

Omitting `SourceShift` from the same request gives the same finite expression but the ordinary remainder scale:

```
control = AsymptoticAnalysis`AsymptoticExponentialCoreInverse[
  Exp[x], 1, {x, Infinity}, {y, 1}
];
(* Under y > 2:
   Normal[control]                  = Log[y] - 1/y
   control["RemainderScaleExpression"] = 1/y^2
*)
```

Thus this witness is not an assertion that the inverse coefficient $-1/y$ is wrong. The incorrect part is the claimed asymptotic order of what remains. A comparison that checks only `Normal` would miss the defect entirely.

2.3 The stored target domain does not exclude the witness

The returned domain requires, in equivalent mathematical notation,

$$e^{|y|}y > 1, \qquad \log(e^{|y|}y) \in \mathbb{R}, \qquad \log(e^{|y|}y) > 1, \qquad \log(e^{|y|}y) > 0.$$

Every condition holds for real $y > 2$. Consequently, the error cannot be dismissed as evaluation outside the stored domain. The local inverse coordinate is

$$v_0 = y + \log y,$$

which is positive and tends to infinity. The failure persists on the declared positive divergent approach.

The use of `Abs[y]` is deliberate. A plain symbolic shift $-y$ might encounter a realness refusal when the target has not yet been constrained by a parameter assumption. The actual native primitive checks showed that `Element[Abs[y], Reals]` and `Exp[-Abs[y]] > 0` simplify to `True`. This exposes the missing target-independence condition without needing to place the target inside `Assumptions`, which the constructor already forbids. The distinction between membership and syntactic dependence is important: realness is not constancy [11, 10].

2.4 Why the remainder is mathematically false

For $y > 1$, the ordinary convergent logarithm identity gives

$$\log(y-1) = \log y + \log(1 - y^{-1}) = \log y - \frac{1}{y} - \sum_{k=2}^{\infty} \frac{1}{ky^k}.$$

Therefore the positive approximation error is

$$E_1(y) = \left(\log y - \frac{1}{y}\right) - \log(y-1) = \sum_{k=2}^{\infty} \frac{1}{ky^k} \sim \frac{1}{2y^2}. \tag{5}$$

In particular,

$$\frac{E_1(y)}{e^{-2y}y^{-2}} \geq \frac{1}{2}e^{2y} \longrightarrow \infty.$$

There is no fixed constant C and no sufficiently large threshold for which $E_1(y) \le Ce^{-2y}y^{-2}$. This is a wrong big-$O$ statement, not a bound with an inconvenient hidden constant.

y	True error $E_1(y)$	Reported scale	Error / scale
5	$2.31436 \cdot 10^{-2}$	$1.81600 \cdot 10^{-6}$	$1.27443 \cdot 10^{4}$
10	$5.36052 \cdot 10^{-3}$	$2.06115 \cdot 10^{-11}$	$2.60074 \cdot 10^{8}$
20	$1.29329 \cdot 10^{-3}$	$1.06209 \cdot 10^{-20}$	$1.21769 \cdot 10^{17}$
50	$2.02707 \cdot 10^{-4}$	$1.48803 \cdot 10^{-47}$	$1.36225 \cdot 10^{43}$

The numerical rows are independent evaluations of the proved formulas using 90-digit arithmetic; they are not four further package runs. Their full values and this provenance distinction appear in `evidence/numerical_samples.csv`. The proof of divergence, rather than the sample values, establishes the defect.

2.5 Severity and affected contract

The appropriate severity is high mathematical correctness risk. The request obtains a normal successful analytic result with an error scale that is exponentially too small. Downstream reasoning that uses that scale could infer nonexistent precision. This report does not claim a particular downstream certificate is already fooled: the bad remainder itself is the reproduced failure.

The input that triggers the defect is avoidable. It uses a target-dependent source shift, whereas the majorant explicitly states a fixed-data premise. That makes a small admission repair possible. It does not make the current successful result sound. A public constructor must either enforce that premise or return a weaker contract that it has actually established.

3 Source-level cause and data flow

3.1 The guard tests the wrong independence set

At the audited source revision, the guard at lines 95–97 of `ExponentialCorePerturbation.wl` checks the following condition [7]:

```
If[x === y || ! FreeQ[{core, perturbation}, y] ||
   ! FreeQ[ass, x | y] || ! FreeQ[{shift, requested}, x],
  fail["InvalidVariables", ...]
];
```

The forward core and perturbation must be target-independent, and the assumptions must concern neither source nor target. However, `shift` and `requested` are grouped together and checked only for the source variable.

Those two fields have different mathematical roles. The requested `CoreInverse` is an expression in the target and must be allowed to contain it. A fixed source shift is a parameter of the source chart and must not contain the target. Sharing one independence check between those fields leaves a hole. `FreeQ` is precisely the syntactic dependency test being used here; the fix does not require a new theorem prover [10].

3.2 The coordinate builder accepts the moving shift

At a source infinity, `exponentialCoreCoordinates` constructs

$$x = \sigma + v$$

for the positive source side. Candidate shifts are required to be exact, source-independent, and provably real. The target is not passed as an exclusion variable to that check. With $\sigma = -|y|$ and original core e^x, normalization produces

$$e^{v-|y|} = A(y)e^v, \qquad A(y) = e^{-|y|}.$$

The Lambert exponential-term parser correctly separates the constant-in-v part of the exponent from its v-dependent part. It records that first factor as the amplitude. This parser is not making an algebraic mistake: $A(y)$ really is constant with respect to v. It is not constant along the eventual target approach [8].

This distinction is the causal point. The helper's local variable discipline is weaker than the constructor's global asymptotic parameter discipline. A source-local parser cannot decide which other symbols the caller intends to vary unless the constructor supplies or enforces that information.

3.3 Exact inversion and coefficients still work

For $b = 0$ and $c = p = 1$, the exact local inverse constructed by the code is

$$v_0 = \log(y/A(y)), \qquad x_0 = \sigma(y) + v_0 = \log y.$$

On the positive target approach this identity is correct, even for the moving chart. For the constant perturbation 1, the first coefficient in the code's normalized exponential sector is

$$C_1 = -\frac{1}{A(y)}, \qquad \chi = \frac{A(y)}{y}.$$

The displayed correction is their product,

$$C_1\chi = -\frac{1}{y}.$$

The target dependence cancels exactly in the finite expression. A test of the core inverse identity, or of the first finite coefficient, consequently gives a reassuring but incomplete result.

3.4 The majorant does not carry the cancelling amplitude

The constructor sets $d = k = 0$ for the constant perturbation and uses $r = p = 1$ for this source chart. Its remainder-scale formula reduces to

$$B_N = \chi^{N+1} = \left(\frac{A(y)}{y}\right)^{N+1}.$$

Its fixed-data majorant may absorb powers of A^{-1} into an existential constant when A is genuinely fixed. Once A depends on y and tends to zero, that absorption is invalid. For the observed $N = 1$ case, $A = e^{-y}$ and the code reports e^{-2y}/y^2.

The exact core identity, the positivity tests, and the coefficient recurrence are therefore compatible with the wrong result. The failed premise is constancy of normalized core data on the target approach, not correctness of exponential algebra. This is why adding another numerical residual check to the finite approximation would not repair the construction theorem.

4 An all-orders analysis of the failure

4.1 A slightly more general exact family

Consider fixed real constants

$$a > 0, \qquad c > 0, \qquad \delta > 0,$$

and the equation

$$ae^{cx} + \delta = y, \qquad y > \delta. \tag{6}$$

Its exact inverse is

$$x_*(y) = \frac{1}{c} \log \frac{y - \delta}{a}.$$

Introduce a real source translation $x = \sigma + v$. With

$$A = ae^{c\sigma}, \qquad v_0 = \frac{1}{c} \log \frac{y}{A}, \qquad \chi = e^{-cv_0} = \frac{A}{y},$$

the zero-sector reconstruction is always

$$\sigma + v_0 = \frac{1}{c} \log \frac{y}{a}.$$

For fixed σ, this is an ordinary fixed-parameter reparameterization. For moving $\sigma(y)$, it is still an exact algebraic identity at each admissible target, but fixed-parameter estimates require reexamination.

Proposition 1 (Specialization of the implemented coefficient recurrence)**.** *For the constant perturbation δ, the code's recurrence with $b = 0$, $p = 1$, and $x = \sigma + v$ gives*

$$C_n = -\frac{\delta^n}{cnA^n}, \qquad n \geq 1.$$

Consequently $C_n\chi^n = -\delta^n/(cny^n)$, independent of the source translation.

Proof. The denominator in the recurrence is Ac and the reconstruction derivative is one. Before the repeated derivatives, the internal quantity is $q_0 = \delta^n/(Ac)$. At step j, for $1 \leq j \leq n - 1$, it is independent of v, so the recurrence acts by

$$q_j = \frac{D_v q_{j-1} - jcq_{j-1}}{Ac} = -\frac{j}{A} q_{j-1}.$$

Thus

$$q_{n-1} = (-1)^{n-1} \frac{(n-1)!\delta^n}{cA^n}.$$

Multiplication by the final $(-1)^n/n!$ gives the stated coefficient. The sector product follows by substituting $\chi = A/y$. □

This calculation follows the recurrence read in `exponentialCoreCoefficient`, rather than inferring an all-orders implementation from a single native output [7]. The independent Python implementation checks this specialization through $n = 7$. It also checks a nonconstant perturbation $v + 1$ through order three by substitution into the original marker-dependent equation. Those finite checks corroborate the transcription; the displayed induction is the all-orders proof for the constant family.

Theorem 1 (True tail and the moving-chart mismatch)**.** *For every fixed integer $N \geq 0$, define*

$$x_N(y) = \frac{1}{c}\log\frac{y}{a} - \frac{1}{c}\sum_{n=1}^{N}\frac{\delta^n}{ny^n}.$$

For $y > \delta$, the error is positive and satisfies

$$\frac{\delta^{N+1}}{c(N+1)y^{N+1}} \leq x_N(y) - x_*(y) \leq \frac{\delta^{N+1}}{c(N+1)y^{N+1}(1-\delta/y)}. \tag{7}$$

In particular,

$$x_N(y) - x_*(y) \sim \frac{\delta^{N+1}}{c(N+1)y^{N+1}}.$$

If the constructor's specialized scale is $B_N = (ae^{c\sigma(y)}/y)^{N+1}$ and $\sigma(y) \to -\infty$, then its ratio to the true error cannot be bounded below by a positive constant: more explicitly,

$$\frac{x_N(y) - x_*(y)}{B_N} \geq \frac{\delta^{N+1}}{c(N+1)a^{N+1}}e^{-c(N+1)\sigma(y)} \longrightarrow \infty.$$

Proof. Put $t = \delta/y$, so $0 < t < 1$. Subtract the exact inverse and use the convergent series for $-\log(1-t)$:

$$x_N - x_* = \frac{1}{c}\sum_{n=N+1}^{\infty}\frac{t^n}{n}.$$

The first term gives the lower bound. For the upper bound, replace each $1/n$ by $1/(N+1)$ and sum the geometric series. The squeeze gives the asymptotic equivalent. Division by B_N gives the final inequality. □

4.2 What the theorem does and does not prove about the package

The native public reproduction was made at depth one with $a = c = \delta = 1$. The all-orders conclusion follows separately from the read constructor and coefficient formulas. It is not an assertion that a completed native depth sweep was obtained: that attempted sweep returned a service error. This distinction matters even when the general mathematics is elementary.

The theorem also does not say that all target-dependent coordinates are unsound in principle. A moving-coordinate theory can be correct when its parameter variation is included in the estimate. It says that the fixed-data estimate currently attached to these admitted requests does not establish their claimed tails.

4.3 Fixed shifts are a necessary positive control

For a fixed shift σ_0, the returned scale differs from the unshifted scale by the fixed factor $e^{c(N+1)\sigma_0}$. Such a factor may legitimately be absorbed in a big-O constant. Therefore a regression test should not demand literal equality of the two remainder expressions for every fixed shift.

For example, with $a = c = \delta = 1$, $N = 1$, and $\sigma_0 = -3$, the specialized scale is e^{-6}/y^2. That remains an $O(y^{-2})$ scale. Confusing this harmless constant change with the invalid e^{-2y}/y^2 change would produce a patch that rejects valid requests or tests the wrong invariant.

5 Minimal repair and its validation

5.1 Separate the two scope contracts

The tested replacement changes just the final clause of the admission guard:

```
(* Before *)
! FreeQ[{shift, requested}, x]

(* After *)
! FreeQ[shift, x | y] || ! FreeQ[requested, x]
```

The rest of the constructor is left unchanged. In particular, `CoreInverse -> Log[y]` remains legal. Replacing the original check with `FreeQ[{shift,requested},x|y]` would be an overcorrection, because an inverse expression is supposed to depend on its target.

The one-line diff is supplied as `code/source-shift-guard.diff`. The Python utility `code/patch_source_shift.py` implements the same replacement. It requires exactly one original anchor and refuses an already patched or ambiguous input. By default it emits a diff. With `-output`, it creates a separate file and refuses an existing destination or the original source path. It is a narrow patch utility for an inspected revision, not an arbitrary-version source rewriter.

5.2 Fresh native observations after the change

The patched native runs downloaded the pinned standalone text, replaced exactly one occurrence of the guard, wrote a temporary copy, and loaded that copy. They did not modify the repository. The successful observations were:

Case	Expected	Observed
Target-dependent shift $-\|y\|$	failure	`Failure`
Supplied inverse `Log[y]`	accepted	`GeneralizedSeries`
Automatic source shift	accepted	`GeneralizedSeries`
Fixed shift -3	accepted	`GeneralizedSeries`
Fixed parameter shift b, $b \in \mathbb{R}$	accepted	`GeneralizedSeries`
Target renamed to z, shift $-\|z\|$	failure	`Failure`
Target z, independent parameter shift $-\|y\|$	accepted	`GeneralizedSeries`

The first failure retained the existing `InvalidVariables` tag. These seven observations classify the result heads; they are not seven complete coefficient-and-tail regression suites. The baseline expression and remainder projections were obtained separately on the unpatched package. The record is a transparent transcription of connector outputs, not an automatically exported MUnit report.

5.3 Documentation and diagnostics

A small follow-up should state the scope directly in the constructor's usage text: a source shift is fixed with respect to both the source and the requested target, though it may depend on fixed external parameters. The current generic message groups `CoreInverse` and `SourceShift` together, which obscures their different dependency rules [7].

The minimal tested patch preserves that message and tag to avoid silently claiming validation of a larger behavior change. A later diagnostic-only change could attach the offending option and target symbol to a dedicated failure. That change should have its own expected-message tests. It is not necessary to establish the mathematical repair.

A defense-in-depth check can also assert that normalized core parameters are free of the target before constructing coefficients and the majorant. This would catch a future chart-building path that accidentally bypasses option admission. It is a postcondition on this specific construction, not a proposal to redesign every option or introduce a general request registry.

5.4 Why the repair is placed before coefficient construction

The invalid request can be identified from its admitted option values before constructing a local chart, evaluating special inverses, or generating sector coefficients. There is no benefit in waiting until a numerical evaluation or a downstream series operation to refuse it. The source shows that the guard precedes those computations [7].

This gives an operational advantage without requiring a benchmark claim: the repaired request exits before entering the coefficient path. No runtime speedup, memory reduction, or broad performance comparison was measured. The existing performance backlog is not repackaged as a new contribution here.

6 Regression artifacts and independent verification

6.1 Executed independent tests

The accompanying Python tests completed successfully: 20 test methods, consisting of 12 mathematical-oracle methods and eight patch-fixture methods. Their internal parameter subtests are not added to the method count. The command and raw output are recorded in `evidence/independent-tests.txt`; environment versions and the count split are in `evidence/independent_tests.json`.

The mathematical checks cover the constant-perturbation coefficient recurrence, amplitude cancellation in each displayed sector, direct residual cancellation for a nonconstant perturbation through order three, logarithmic finite coefficients, exact rational tail bounds, scaled equations, moving-shift scales, fixed-shift constant ratios, and the growing counterexample ratio. The domain-validation tests reject invalid depths and invalid positive-tail parameters.

The rational bounds in (7) are computed using Python's exact `Fraction` type. High-precision numerical values are used only to check independent samples against those bounds. The tests do not use a fabricated copy of a package result as their mathematical oracle. Conversely, they do not execute Wolfram Language and therefore cannot establish Mathics or Wolfram package compatibility.

The patch tests check single-anchor replacement, refusal on missing or duplicated anchors, refusal on repeated application, preservation of unrelated text and line endings, creation of a separate copy, and refusal to overwrite either an existing destination or the source. Those tests are properties of the supplied Python utility, not additional reproductions of N01.

6.2 Supplied Wolfram tests

`code/ReviewSourceShift.wlt` contains twelve MUnit cases for a repaired package. Besides the observed outcome controls, it includes positive and slowly moving target shifts, preservation of ordinary finite coefficients, the existing rejection of a source-dependent shift, and the fixed-shift remainder factor. This complete MUnit file was not executed during the audit.

`code/ProbeSourceShift.wl` records the bad request and its unshifted control without modifying package definitions or downloading anything. On the baseline it should expose the incorrect remainder. On a repaired package it should instead show a failure for the bad request. It is an observation script, not a test that treats either outcome as a success.

`code/PortableSourceShift.wl` supplies a smaller seven-case result-head smoke test without an MUnit dependency. A package must be loaded before the file is evaluated. The script distinguishes setup absence and unexpected result heads, and it aborts after printing a failure summary when an expected outcome is not obtained. The corresponding individual classifications were observed in Wolfram; the complete supplied file was not itself used as the receipt and has not been run under Mathics.

6.3 A useful nontrivial test invariant

A finite-expression-only regression is insufficient. For fixed source translations in the exact family (6), the finite inverse approximation must agree after reconstruction, while the remainder scales need only be equivalent up to target-independent factors. For moving translations, the current fixed-data constructor should refuse the request. These are three different expectations, not a single string-equality test.

The retained first omitted sector can provide a useful diagnostic in this family. Its actual contribution has magnitude $\delta^{N+1}/(c(N+1)y^{N+1})$, whereas the faulty reported scale has an extra factor $A(y)^{N+1}$. Their ratio exposes the same unbounded factor. This is a targeted diagnostic for the proved family, not a universal rule that every first omitted coefficient is automatically a lower bound for every asymptotic remainder.

7 Wolfram and Mathics: precise portability conclusions

7.1 What is established for Wolfram 15

The unmodified public constructor was loaded and executed in Wolfram 15.0.0 on Linux. The actual wrong result and the correct unshifted control were projected under the same condition $y > 2$. The minimal guard was subsequently exercised in two successful patched calls covering the seven classifications above. These are current observations for the pinned source and the reported runtime.

They do not establish performance equivalence, behavior of every supported special-function family, a complete native test pass, or preservation of every failure-message ordering. The supplied acceptance tests intentionally go beyond the small observed native matrix. In particular, a result head remaining `GeneralizedSeries` is weaker evidence than proving its full expression and remainder unchanged.

7.2 What remains unestablished for Mathics3

The faulty guard and the repaired predicate are in shared construction code, not in a Mathics-only override. The syntactic repair uses the same existing `FreeQ` idiom as the surrounding code. That makes it a natural common repair. It does not prove that the unpatched Mathics runtime accepts this particular `Abs[y]` witness: a compatibility realness or simplification path could conservatively refuse it before a result is produced.

No Mathics installation was available for local execution, and no successful Mathics run was obtained through another service. Consequently, the report does not attribute the observed Wolfram output to Mathics. A conservative Mathics refusal would not refute the shared source defect; it would show that this witness is masked there by an earlier runtime-specific limitation. Equally, a passing syntax-only guard test would not certify the remaining analytic paths.

The portable smoke should be run against both canonical and generated entries after the patch is integrated. Its two most important role controls are the renamed target and the independent parameter whose printed name is `y`. The contract is about which symbol is the requested target, not about a hardcoded spelling. The separate `CoreInverse -> Log[y]` control prevents an apparently stronger but incorrect guard from being accepted.

8 Broader source pass: what was not promoted into new findings

8.1 Special-function formulas and domain checks

The reviewed Gamma and Barnes paths explicitly combine positive factors through logarithmic carriers, and the Barnes code retains Bernoulli-tail information instead of identifying a finite asymptotic model with an exact function. The Dirichlet paths attach explicit integral-comparison or geometric-moment bounds. The parameterized special-function path separates certain noninteger leading powers from analytic bodies and performs a separate real-domain check [2, 9].

Those observations explain why this audit examined parameter and coordinate scope rather than only coefficient arithmetic. They are not a certification that all adapters are correct. No additional distinct defect in those inspected formulas is asserted here. Where a candidate concern matched a recorded mechanism, it was excluded rather than given a new identifier.

8.2 Series operations and certificates

Selected representation, arithmetic, refinement, and interval-certificate paths were inspected to understand how assumptions and error scales can become reusable data. That inspection reinforces the importance of repairing N01 at construction: an object with a false error scale is already unsound before any consumer decides whether it can use that scale.

The witness in this report does not pass through `InverseCertificate`, a native `Series` fallback, numerical root-finding, or an externally forged result association. No claim is made that those separate facilities share the new bug. Likewise, the exact inverse (2) is used as an independent oracle; it is not a value extracted from the certificate engine.

8.3 Performance, API, and missing-feature requests

The user's review request includes performance, API design, and future development. A novelty-constrained audit should not manufacture a fresh optimization list when the repository already records broad proposals in those areas. This report therefore makes no new benchmark or generic cache recommendation. The concrete API defect is the mismatched scope contract of two options; the concrete efficiency recommendation is to reject that invalid scope before expensive chart and coefficient work; and the concrete development opportunity is the narrow, proved extension in the next section.

The lack of an additional promoted finding in a reviewed area is not a statement that the area is defect-free. It means this audit did not obtain a sufficiently distinct and well-supported new claim there. The source inventory and evidence boundaries are included so the next reviewer can continue from actual inspected paths rather than an implied exhaustive certification.

9 A narrowly scoped development opportunity

9.1 A chart-independent reference representation

For the exact family (6), a representation based on $\eta = \delta/y$ avoids the spurious source-shift dependence completely:

$$x_N(y) = \frac{1}{c}\log\frac{y}{a} - \frac{1}{c}\sum_{n=1}^{N}\frac{\eta^n}{n}, \qquad |x_N - x_*| \le \frac{\eta^{N+1}}{c(N+1)(1-\eta)}.$$

This is a fully specified small extension, not a promise to support arbitrary moving-coordinate transseries. It identifies a family, an admitted real domain, exact coefficients, an explicit remainder bound, and an independent exact inverse.

Another way to express the same normalization is to store $D_n = A^n C_n$ with the target sector $1/y$. Proposition 1 gives $D_n = -\delta^n/(cn)$, so all source-shift dependence disappears before numerical substitution. This algebraic cancellation is why the finite result can be right even when the original error norm is wrong.

For a future implementation that deliberately supports moving source shifts in this family, the shift would be treated as an auxiliary exact coordinate, not as a hidden fixed parameter. The remainder theorem above supplies the required replacement contract. Outside such a proved family, the minimal refusal remains the preferable behavior.

9.2 Useful acceptance questions for that extension

A candidate implementation should answer three concrete questions. Does the reconstructed finite approximation agree for every admitted real shift? Does its quantitative remainder depend only on the original equation's fixed data and target, rather than on an arbitrary moving chart? Does it reject or separately analyze inputs outside $a, c, \delta > 0$ and $y > \delta$, rather than extending the positive-tail proof by analogy?

A further sign-generalization is possible, but it requires its own statement. For negative perturbations the logarithmic series alternates, so the positive lower bound in (7) is not the right contract. The upper absolute bound based on $|\delta/y| < 1$ has a different proof statement. The supplied reference intentionally validates only the positive-tail domain; it does not silently claim all real parameters.

This narrow route provides a useful testbed for future chart-normalization work. It is not necessary for the immediate repair and should not delay rejecting the currently unsupported target-dependent shifts.

10 Recommended disposition

Record N01 as a reproduced public wrong-remainder defect at the pinned revision. Apply the tested split scope guard to the canonical source, regenerate the standalone through the repository's maintained generation process, and execute the supplied focused tests on the intended runtimes. Keep the native baseline observation, the patched result-head observations, independent mathematical tests, and any subsequent Mathics acceptance as separate records.

The central correctness obligation is small and explicit:

$$\mathrm{FreeQ}(\mathrm{SourceShift}, \{x, y\}) \quad \text{but only} \quad \mathrm{FreeQ}(\mathrm{CoreInverse}, \{x\}).$$

The actual Wolfram implementation uses `x | y` as the pattern alternative, as shown in the patch. The set-like notation above summarizes the forbidden dependency roles; it is not proposed Wolfram syntax.

No broader result-schema or coefficient-engine rewrite is needed to close the observed route. Documentation should make the two option roles visible. A later target-free normalized-parameter postcondition is a useful local safeguard. Deliberate moving-chart support should be tied to a theorem such as Theorem 1, not inferred from cancellation in a finite expression.

A Reproduction and integration workflow

A.1 Inspect and patch a local copy

From the supplied archive, the patch utility can emit a diff against the canonical source of a separate repository checkout:

```
python code/patch_source_shift.py \
  /path/to/Asymptotic/src/Kernel/ExponentialCorePerturbation.wl
```

To create a separate patched copy for inspection, add `-output` with a new filename. The utility never overwrites the original. The supplied diff has the same guard replacement and is intended for the pinned source. Review the actual diff before integrating it with any newer revision.

The native patched observations used a temporary modified standalone copy to make the experiment self-contained. For upstream development, edit the canonical module and regenerate the distribution rather than maintaining an independent manual edit of the generated root entry. The repository documents the canonical/generated distinction [2, 1].

A.2 Run the native or portable probes

In a fresh kernel, first load the chosen package entry, then read the probe:

```
Get["/path/to/AsymptoticAnalysis.wl"];
Get["/path/to/asymptotic_audit/code/ProbeSourceShift.wl"];
```

For patched Wolfram acceptance, load the same entry and use:

```
TestReport["/path/to/asymptotic_audit/code/ReviewSourceShift.wlt"]
```

For the portable result-head smoke, use `Get` on `PortableSourceShift.wl` after loading the chosen entry. The scripts contain no automatic downloads. Pinning, runtime identification, and selection of the correct entry remain explicit in the surrounding run.

A.3 Run the independent checks and build the article

```
cd code
python -m unittest discover -s . -p 'test_*.py' -v
cd ../article
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
```

The Python mathematical oracle requires SymPy and mpmath. Exact versions used in this audit are recorded in the independent evidence JSON. The article builds from one TeX source and standard LaTeX packages; no downloaded font files or external figures are needed.

B Artifact map and evidence boundaries

Artifact	Purpose and boundary
`article/review.tex`, `review.pdf`	Article source and compiled report.
`code/source-shift-guard.diff`	Minimal shared-source guard change; corresponding replacement exercised natively.
`code/patch_source_shift.py`	Exact-anchor diff/copy utility; eight independent fixture methods passed.
`code/ProbeSourceShift.wl`	Records baseline or repaired public behavior without mutating definitions.
`code/PortableSourceShift.wl`	Seven-case outcome smoke; not executed under Mathics in this audit.
`code/ReviewSourceShift.wlt`	Twelve supplied MUnit acceptance cases; complete file not executed here.
`code/reference.py`	Independent recurrence specialization and exact rational bounds. Not a package replacement.
`code/test_reference.py`, `test_patch.py`	Twenty executed independent Python test methods in total.
`evidence/native_observations.json`	Explicit manual transcription of successful connector observations and their limited scope.
`evidence/independent-tests.txt`, `independent_tests.json`	Raw independent test log, environment and count split.
`evidence/numerical_samples.csv`	Independent evaluations of the proved error and claimed scale.
`evidence/source_inventory.csv`	Inspected source paths, complete/partial scope, and the claims actually made.

Artifact	Purpose and boundary
`evidence/novelty_ledger.csv`, `scope.json`	Nearest prior identifiers, novelty boundary, execution limits, and pinned revision.

The archive contains no checksum files. The revision identifier is provenance for the reviewed source, not a checksum manifest. No repository changes were pushed or saved by this audit.

References

[1] Vladimir Reshetnikov, *Asymptotic*, pinned repository and README, revision `8cee870994f506b501bae3ea6bd4a3a7edb895c1`.
https://github.com/VladimirReshetnikov/Asymptotic/tree/8cee870994f506b501bae3ea6bd4a3a7edb895c1

[2] *Kernel source map*, same revision, `src/Kernel/README.md`.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/README.md

[3] *Code review status register*, same revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/docs/development/CODE_REVIEW_STATUS.md

[4] *Wave 3 intake*, same revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/docs/development/WAVE_3_INTAKE.md

[5] *Wave 4 intake*, same revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/docs/development/WAVE_4_INTAKE.md

[6] *Wave 5 review index*, same revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/wave-5/README.md

[7] *ExponentialCorePerturbation.wl*, same revision. Key sites: `exponentialCoreCoordinates`, `exponentialCoreExactInverse`, `exponentialCoreCoefficient`, and `exponentialCoreConstruct`; admission at lines 95–97 and sector construction at lines 113–127.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/ExponentialCorePerturbation.wl#L95-L127

[8] *LambertInverse.wl*, same revision; `lambertExponentialTerm` and `lambertExponentialCore`.
https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/LambertInverse.wl

[9] Special-function modules at the same revision: `GammaForward.wl`, `BarnesForward.wl`, `DirichletSpecialFunctions.wl`, and `ParameterizedSpecialFunctions.wl`; see the source inventory for scope.
https://github.com/VladimirReshetnikov/Asymptotic/tree/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel

[10] Wolfram Research, *FreeQ*, Wolfram Language documentation, accessed 10 September 2026.
https://reference.wolfram.com/language/ref/FreeQ.html

[11] Wolfram Research, *Element*, Wolfram Language documentation, accessed 10 September 2026.
https://reference.wolfram.com/language/ref/Element.html